

Phase 4 System Architecture

Preferred covered reading edition

APPROVED SCHEDULING AMENDMENT | PRESERVED REVIEW FOLLOWS

Read amendment first; underlying phase approval status is retained

Aesthenda Core Phase 4 Scheduling Amendment

Status: APPROVED DECISION PROPAGATION

Effective: September 7, 2026

This amendment applies DG-02, DG-03, and the approved overlap presentation to Phase 4. It controls conflicting scheduling language in the preserved review package; it does not grant new approval to unrelated material.

Fixed booking holds

All businesses use a fixed ten-minute booking hold, timed by the server from hold creation. There is no business, Service, or Appointment duration setting. Release early on explicit cancel/close; otherwise expire automatically. Confirmation revalidates the hold and current scheduling state; an expired hold cannot confirm from stale client state.

Professional overlap exceptions

Public booking never offers or confirms a non-fitting hands-on overlap. The authenticated owner-professional may manually book it after a specific warning and explicit confirmation bound to the exact request and current affected revisions. Published policy alone cannot authorize a hands-on conflict. Record the warning, actor, time, affected Appointments and segments, policy version, confirmation, and outcome. Preserve the full booked duration unless the professional explicitly edits it; never silently shorten work or move an existing appointment.

Minimal calendar presentation

Shade only the exact conflicting interval amber on both affected appointments, keeping their normal colors elsewhere. Show the same highlight while placing or moving the appointment, before the confirmation warning, and retain it after approval while the conflict exists. Use no persistent overlap label or extra icon. Tap or click reveals the overlap duration, affected appointments, and approval details. Provide the same details through keyboard interaction and screen-reader text without adding visible clutter.

Approved example

With zero buffers and other rules satisfied, a 10:00-10:15 application, 10:15-10:45 released processing segment, and 10:45-11:30 haircut permit a fitting 30-minute public service in the processing gap. A 45-minute service at 10:15 ends at 11:00 and is not offered publicly. A manual booking previews amber from 10:45 to 11:00 on both appointments, warns of the 15-minute overlap, and requires Book anyway before saving. Go back returns to editing without committing. Approval preserves the 45-minute booking and existing haircut timing.

Architecture effects

Scheduling owns public fit, hold expiry, and manual exception evaluation. UI, agents, integrations, and booking orchestration call the same canonical actions. A professional-only exception does not weaken public availability or authorize another actor.

Preview returns exact conflict intervals and affected revisions. Save re-evaluates under scheduling locks and commits the appointment, confirmation evidence, audit, and outbox consistently. A changed request or affected revision invalidates the prior confirmation. Booking-hold time is a platform safeguard, not business policy. DG-01 removes all Resource tracking, capacity, assignment, and resource-fit implications from the preserved Phase 4 review.

Verification requirements

Verify ten-minute holds across businesses, early release, stale expiry rejection, public omission/rejection of the 45-minute example, rejected unconfirmed manual save, accepted confirmed save, unchanged durations, and rejection of stale confirmation after a concurrent revision. Verify exact interval rendering before and after approval with details available on demand. These are design acceptance requirements; no application implementation is present in this document library.

AESTHENDA CORE

Phase 4 - System Architecture

Review Package

ONE BOOKING BRAIN. MANY CONTROLLED INTERFACES.

Purpose

Define the architectural boundaries that allow Aesthenda to remain simple for one independent professional while supporting intentional double-booking, configurable policies, brand-flexible booking pages, payments, integrations, and future intelligence.

Document status

Proposed architecture. Approval authorizes Phase 5 database design; it does not authorize production implementation.

Prepared: August 31, 2026

1. Architecture Decisions at a Glance

STATUS: PROPOSED - REQUIRES APPROVAL

ID	Proposed decision	Why
A-001	Begin as a modular monolith.	Keeps one coherent transaction and deployment boundary while preserving clear internal modules.
A-002	Use one canonical application-action layer.	Prevents web, mobile, AI, voice, and integrations from duplicating business rules.
A-003	Use a relational transactional database as the source of truth.	Scheduling, payments, policies, and audit require strong consistency and constraints.
A-004	Model active time, processing time, professional capacity, and resource capacity separately.	Makes intentional double-booking natural instead of a calendar workaround.
A-005	Use transactional outbox plus background workers for external effects.	Prevents committed bookings from losing notifications or integration updates.
A-006	Store booking-page themes as validated, versioned configuration.	Enables expressive branding without forking booking logic.
A-007	Use adapters for Square, messaging, calendar, storage, and future providers.	Keeps third-party behavior outside the core domain.
A-008	Agents receive scoped actions, never database credentials.	Keeps automation within the same permissions and policy boundaries as humans.

Recommended implementation posture

A server-oriented TypeScript application with PostgreSQL is the proposed starting posture. A responsive web client is the first experience surface. A job runner handles reliable asynchronous work; object storage holds approved media assets; a short-lived coordination store may be added only when measured needs justify it. Exact vendors and framework versions remain technology-decision records at implementation time.

Review gate

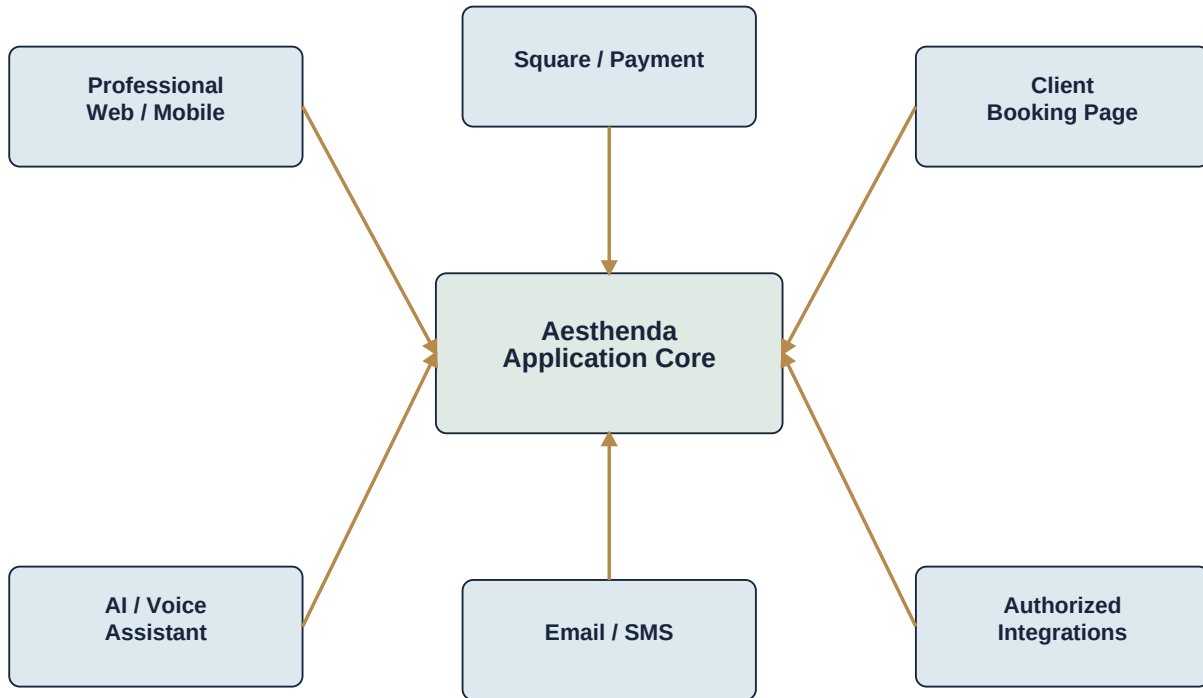
Do these decisions keep v1 understandable and reliable without blocking mobile, AI, integrations, or future team expansion?

2. System Context

STATUS: PROPOSED - REQUIRES APPROVAL

Aesthenda owns canonical appointments, policies, clients, services, availability, invoices, audit history, and the business-facing interpretation of payment and delivery events.

One booking brain; many controlled interfaces

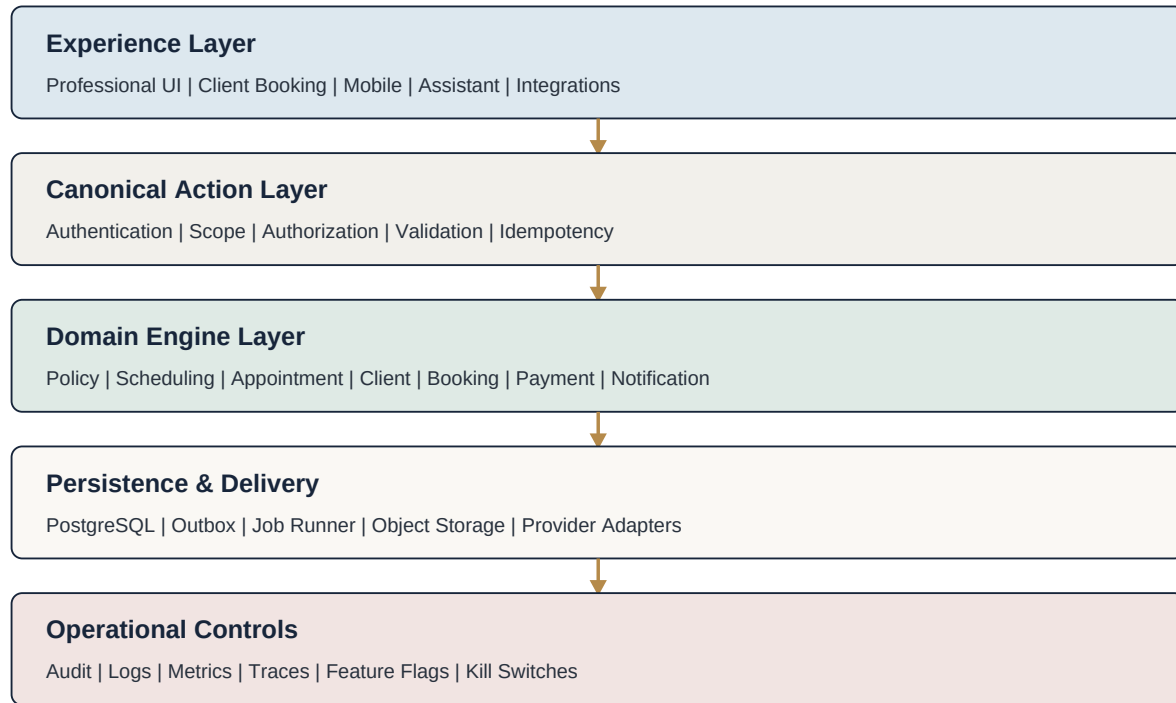


Boundary rules

- Professional and client interfaces never implement scheduling policy independently.
- Square is authoritative for its processor events; Aesthenda verifies and reconciles them.
- Messaging providers deliver communications but do not own notification intent or history.
- Integrations and AI use scoped application actions and receive only the data necessary for the approved task.

3. Architectural Layers

STATUS: PROPOSED - REQUIRES APPROVAL



Dependency rule

Dependencies point inward. Experience and provider adapters may depend on application contracts; domain engines do not depend on pages, web frameworks, AI models, Square-specific objects, or messaging-provider payloads.

Module contract

- Each module owns its rules, application actions, persistence interface, events, and tests.
- Cross-module writes occur through application actions or explicit domain contracts.
- Direct cross-module table mutation is prohibited.
- Read projections may combine data for performance but cannot become alternate write paths.

4. Core Engine Responsibility Matrix

STATUS: PROPOSED - REQUIRES APPROVAL

Engine	Owns	Must not own
Identity & Account	Authentication linkage, business scope, capabilities, sessions	Scheduling or payment policy
Policy	Versioned business settings and effective-value resolution	Interface-specific copies of settings
Scheduling	Availability, timed segments, capacity, slot calculation, holds, conflicts	Appointment lifecycle or payment settlement
Appointment	Appointment state, items, status history, cancellation/reschedule intent	Independent slot algorithms
Client	Profiles, preferences, notes, forms, merge rules	Payment instruments
Booking	Client-facing orchestration of slots, holds, terms, confirmation	Duplicate scheduling rules
Payment	Invoices, payment intent, reconciliation, refunds, external-payment records	Processor secrets outside adapters
Notification	Message intent, templates, preferences, delivery history	Appointment state authority
Theme & Publishing	Validated theme drafts, previews, assets, published versions	Canonical service, price, policy, or availability data
Audit	Attribution and immutable consequential-action record	Operational business mutation
Integration	Connections, scopes, sync cursors, provider adapters	Direct domain writes
Agent Orchestration	Intent translation, clarification, confirmation, scoped action invocation	Canonical facts or direct persistence

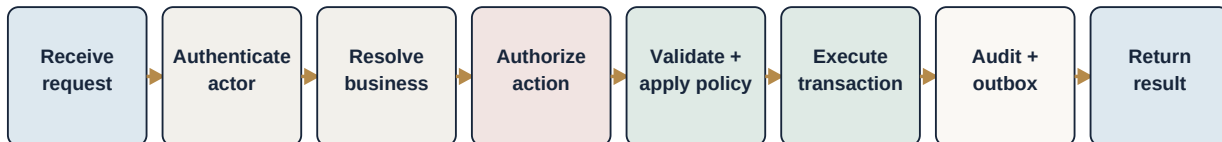
Ownership rule

Every canonical fact has one owning engine. Other engines reference it by stable identifier or consume an explicit event; they do not create shadow truth.

5. Canonical Action Pipeline

STATUS: PROPOSED - REQUIRES APPROVAL

Canonical write-action pipeline



Any failure stops the write, returns a stable error, and records safe diagnostic context.

External delivery work begins only after the database transaction commits.

Action contract

- Input includes actor context, business context, request ID, idempotency key for consequential writes, and typed action data.
- Authorization evaluates capability and record scope before domain execution.
- Policy resolution returns the effective value and source: appointment override, service, location, business, or accepted starting recommendation.
- The domain transaction commits the state change, audit record, and outbox messages atomically.
- The response returns result data, warnings, effective policy, audit reference where appropriate, and stable error codes.

No bypass rule

Internal admin tools, migrations, support operations, AI, and integrations require explicit controlled pathways. Convenience is not permission to bypass canonical actions.

6. Scheduling Time Model

STATUS: PROPOSED - REQUIRES APPROVAL

The scheduling engine uses segments, not one undifferentiated duration. A service template may produce one or more appointment segments.

Concept	Meaning	Example
Active segment	Professional hands-on capacity is occupied.	Color application; haircut; rinse and finish
Processing segment	Professional may be available while the client waits.	Color development
Client occupancy	Client continues to occupy a station, chair, room, or waiting resource.	Client remains at station during processing
Resource reservation	A named or pooled resource is consumed for a time range.	Chair 2; treatment room; dryer
Buffer	Business-configured protected time before or after a segment/service.	Cleanup; preparation
Travel segment	Optional protected time between service locations.	Mobile appointment travel

Capacity dimensions

- Professional capacity: whether hands-on work can overlap.
- Resource capacity: how many clients a station pool, room type, or equipment pool can support.
- Business policy: whether overlap is off, processing-compatible, service-enabled, or approval-based.
- Appointment reality: case-specific override, locked terms, and current status.

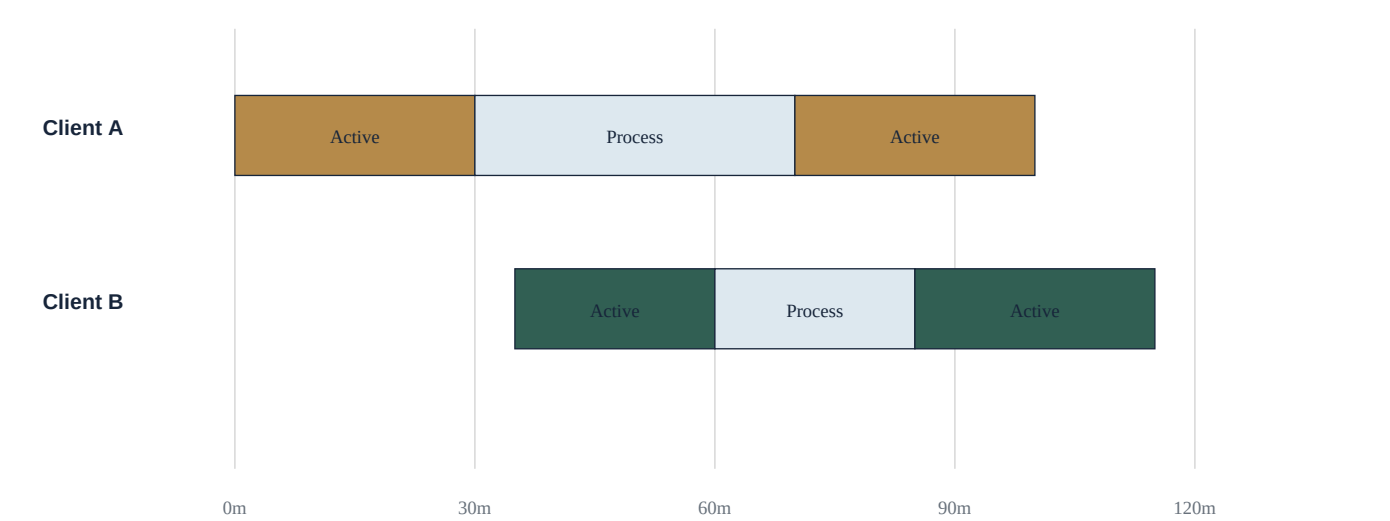
Core invariant

A slot fits only if every active segment and required resource can be satisfied through the entire proposed timeline, including the professional returning for the next hands-on segment.

7. Intentional Double-Booking Model

STATUS: PROPOSED - REQUIRES APPROVAL

Example: compatible processing-time overlap



The engine evaluates professional occupancy and physical-resource occupancy separately.

Supported modes

Mode	Behavior
Off	No overlapping professional occupancy is offered or created without a case-specific override.
Processing-compatible	Offer overlap only inside eligible processing segments when return timing and resources fit.
Selected services	Only explicitly enabled service combinations may overlap.
Professional approval	Show possible overlaps to the professional for one-tap review; do not expose them publicly until approved by policy.
Case-specific override	Professional intentionally places an appointment despite a warning; system records consequences and audit.

Product requirement

Double-booking must feel like a normal capability: stacked calendar display, an Add During Processing shortcut, clear fit/conflict explanation, and no need to falsify duration or availability.

8. Overlap-Aware Slot Calculation

STATUS: PROPOSED - REQUIRES APPROVAL

Calculation sequence

Step	Decision
1. Build timeline	Expand the selected service into active, processing, buffer, resource, and travel segments.
2. Load effective policy	Resolve booking window, notice, overlap mode, service eligibility, buffers, location, and resource rules.
3. Generate candidates	Create start candidates within business availability and client-facing booking windows.
4. Test professional capacity	Reject candidates whose active segments exceed allowed professional capacity.
5. Test resource capacity	Reject candidates whose required station, room, chair, or equipment pool is exhausted.
6. Test return feasibility	Ensure the professional can return for every later active segment without conflict.
7. Test holds and concurrency	Include unexpired temporary holds and lock/serialize final confirmation.
8. Classify result	Return available, available-with-overlap, professional-approval-required, or unavailable with reason.

Concurrency protection

- Availability results are advisory until confirmation.
- Final creation revalidates inside a transaction.
- Conflicting confirmations produce a stable slot-no-longer-available result.
- Idempotency prevents retrying the same request from creating duplicate appointments or payments.

9. Appointment and Booking Orchestration

STATUS: PROPOSED - REQUIRES APPROVAL

Public booking path

- Load published booking configuration and effective client-facing policy.
- Select service and eligible location context.
- Request slot classifications from Scheduling.
- Create a short-lived hold when the client begins confirmation or payment.
- Collect required identity, consent, forms, deposit/payment authorization, and explicit terms acceptance.
- Confirm appointment transactionally; release or expire the hold on failure.
- Emit confirmation, payment reconciliation, notification, analytics, and audit work through the outbox.

Professional booking path

- May see ordinary and overlap-compatible opportunities.
- May use Add During Processing or drag into an overlapping timeline.
- Receives fit explanation, warning level, and affected appointments/resources.
- May confirm an authorized overlap or case-specific override.
- Client-facing confirmation uses the same canonical appointment and saved policy terms.

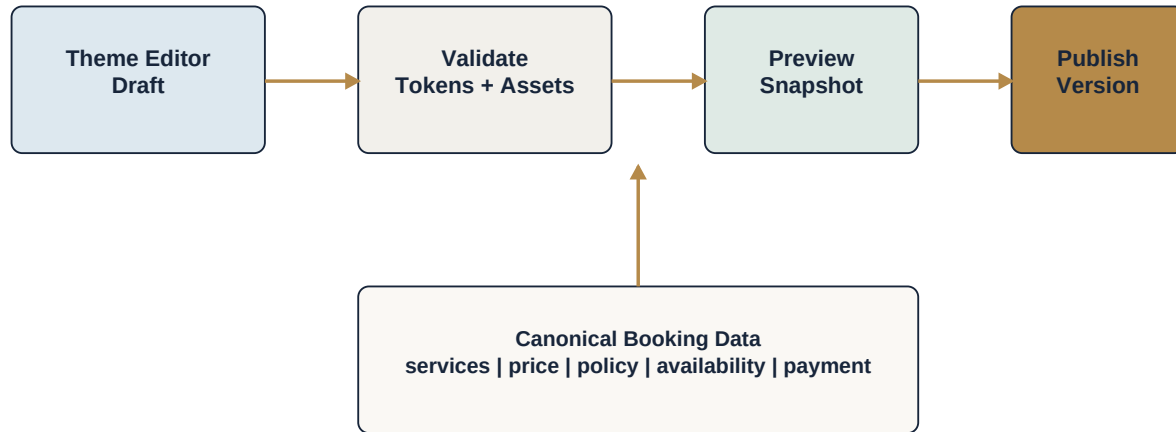
Lifecycle authority

Appointment owns status transitions. Booking orchestrates creation; Scheduling decides fit; Payment decides financial state; Notification records communication. No engine infers another engine's canonical state from UI behavior.

10. Booking-Page Theme and Publishing Architecture

STATUS: PROPOSED - REQUIRES APPROVAL

Brand presentation wraps canonical booking behavior



A failed publication leaves the previous published theme active.

Theme configuration

- Curated theme identifier and schema version
- Color and semantic token overrides within supported ranges
- Approved font pairing identifiers rather than arbitrary executable font sources
- Layout density, card, button, border, and imagery presentation options
- Logo, professional image, and cover-asset references
- Accessibility validation result and warnings
- Draft, preview snapshot, published version, and publication history

Security and integrity

- No arbitrary HTML, JavaScript, CSS, remote font URL, or executable embed in v1.
- Media uses validated type, size, metadata, access, and transformation rules.
- Theme presentation cannot hide or modify canonical price, duration, policy, consent, payment, or availability content.
- Publishing is atomic: a failed publish leaves the previous version active.

11. Identity, Authorization, and Business Isolation

STATUS: PROPOSED - REQUIRES APPROVAL

Identity model

- A human identity may authenticate through an external identity provider, but Aesthenda owns business membership and capability decisions.
- v1 exposes one owner-professional experience; internal capability checks remain explicit.
- Agent and integration identities are separate actors with independently revocable scopes.
- Support access is time-bounded, attributable, least-privileged, and audited.

Authorization sequence

- Validate authenticated actor.
- Resolve business from trusted context, never arbitrary client input alone.
- Confirm membership or scoped connection.
- Evaluate action capability and record-level scope.
- Apply client-data sensitivity rules.
- Record consequential actor, source, request, previous state, result, and reason when required.

Isolation controls

- Business/account identifier on protected aggregates or a provable parent path.
- Repository queries require scope explicitly.
- Database RLS or equivalent defense in depth.
- Signed, short-lived asset access.
- Cross-business administrative operations use separate privileged pathways and monitoring.

Security invariant

Possession of a record identifier is never sufficient authority to read or mutate the record.

12. Payments, Notifications, and Integrations

STATUS: PROPOSED - REQUIRES APPROVAL

Boundary	Synchronous responsibility	Asynchronous responsibility
Square adapter	Create/confirm processor operation when required for booking transaction	Verify signed events, reconcile settlement/refund/dispute state
Notification engine	Create canonical message intent	Render, deliver, retry, record provider result, suppress duplicates
Calendar adapter	Validate connection and accepted sync command	Push/pull changes, maintain cursor, resolve or surface conflict
Object storage	Issue controlled upload/download intent	Scan/transform asset, update readiness state
Analytics	None in the critical booking path	Consume approved events into reporting projections
Agent orchestration	Interpret intent and request clarification/confirmation	Continue only approved durable workflows through canonical actions

Integration rule

Provider payloads are translated at the adapter boundary. Core engines use Aesthenda contracts and never expose processor-specific state as the domain model.

Failure posture

- External timeouts do not cause blind retries of non-idempotent operations.
- Signed webhooks are verified before processing and deduplicated by provider event ID.
- Delivery failures do not roll back an already committed appointment.
- Reconciliation jobs identify and surface mismatched internal/external financial states.

13. Events, Jobs, and Reliability

STATUS: PROPOSED - REQUIRES APPROVAL

Transactional outbox

The same database transaction that commits a consequential state change also writes the events/jobs that must follow. A worker claims them after commit, applies retry policy, and records completion or terminal failure.

Event/job	Typical consumers	Reliability expectation
AppointmentConfirmed	Notification, analytics, calendar integration	At least once; idempotent consumers
AppointmentMoved/Canceled	Notification, calendar integration, payment policy	Ordered per appointment where required
PaymentEventVerified	Payment reconciliation, invoice projection, audit	Deduplicated; signed source retained
PolicyPublished	Booking cache/projection, audit	Versioned; existing appointment terms unchanged
ThemePublished	Booking-page cache/CDN, audit	Atomic version switch; previous version recoverable
AutomationRequested	Agent/automation runner	Capability rechecked at execution time

Retry classes

- Transient: exponential backoff with jitter and bounded attempts.
- Rate-limited: honor provider retry guidance and global safety limits.
- Permanent: stop automatically, preserve evidence, and surface human action.
- Ambiguous external outcome: reconcile before attempting another financial or booking mutation.

14. Audit, Observability, and Operational Control

STATUS: PROPOSED - REQUIRES APPROVAL

Signal	Carries	Must avoid
Audit event	Actor, business, action, target, source, before/after summary, policy version, result	Secrets, full payment data, sensitive note contents
Structured log	Request/correlation ID, module, operation, safe error code, latency	Client text, credentials, raw processor payloads by default
Metric	Rates, latency, conflicts, holds, retries, delivery, reconciliation health	High-cardinality personal identifiers
Trace	Cross-module and provider timing with correlation	Sensitive values in attributes

Required operational views

- Booking funnel and slot-calculation health
- Overlap offer, acceptance, override, and conflict rates
- Temporary hold expiration and confirmation conflicts
- Payment reconciliation and signed-event processing
- Notification delivery and suppression
- Integration connection and sync health
- Agent action, clarification, denial, and kill-switch activity
- Theme publication and asset-processing failures

Control plane

Feature flags gate incomplete capabilities. Separate kill switches can stop automated writes, external sync, or new theme publication while preserving core manual booking operations.

15. Deployment and Scaling Posture

STATUS: PROPOSED - REQUIRES APPROVAL

Initial topology

- One responsive web application and server/API deployment.
- One PostgreSQL primary data store with managed backups and point-in-time recovery capability.
- One background worker deployment using the same application contracts.
- Managed object storage for validated booking-page assets and permitted client media.
- Provider adapters for identity, Square, email/SMS, calendar, and future integrations.
- CDN/cache for public static assets and published theme configuration where safe.

Scaling sequence

Pressure	First response	Only later
Read traffic	Indexes, projections, safe caching, CDN	Read replicas
Slot calculation	Query optimization, bounded search, precomputed availability windows	Dedicated scheduling service after measured need
Background jobs	More workers, partitioned queues, rate controls	Separate worker services by workload
Integration volume	Per-provider throttles and cursors	Independent integration runtime
Global expansion	Explicit time-zone data and regional configuration	Regional data/deployment architecture after legal review

Extraction rule

A module becomes a separate service only when measured scale, fault isolation, security, or ownership requires it and the action/event contract is already stable.

16. Nonfunctional Requirements

STATUS: PROPOSED - REQUIRES APPROVAL

Quality	Initial architectural requirement
Availability	Graceful degradation: core calendar/manual operations remain usable when optional integrations or AI are unavailable.
Consistency	Confirmed booking and payment records use transactional consistency; projections may be eventually consistent and visibly so.
Performance	Set measurable budgets during Phase 7/9 for core professional actions, public slot searches, and publication.
Accessibility	Professional and client surfaces target WCAG 2.2 AA behavior; theme controls enforce safe transactional presentation.
Privacy	Minimize collection, separate sensitive content, define retention, and support authorized export/deletion workflows.
Recovery	Set and test RPO/RTO before launch; restore drills produce evidence.
Explainability	Rule denials, overlap warnings, automation, and payment states provide safe human-readable explanations.
Portability	Core contracts and domain objects do not depend on one UI, AI model, payment payload, or messaging provider.

Performance caution

The blueprint deliberately does not invent numeric service-level objectives before expected traffic, deployment region, and user research are known. Phase 9 must set and approve measurable targets before release.

17. Architecture Decision Records

STATUS: PROPOSED - REQUIRES APPROVAL

ADR	Decision	Status
ADR-001	Modular monolith for v1	Proposed
ADR-002	PostgreSQL as canonical transactional store	Proposed
ADR-003	Canonical action/application layer for all interfaces	Proposed
ADR-004	Segment-and-capacity scheduling model	Proposed
ADR-005	Transactional outbox and background worker	Proposed
ADR-006	Validated versioned booking-page theme configuration	Proposed
ADR-007	Provider adapters for external services	Proposed
ADR-008	Separate agent/integration identities with scoped capabilities	Proposed
ADR-009	Database-enforced business isolation as defense in depth	Proposed
ADR-010	Module extraction only after measured need	Proposed

Technology choices intentionally deferred

- Hosting vendor and region
- Identity provider
- Web framework and exact runtime versions
- Job execution provider
- Object-storage and image-processing provider
- Email/SMS providers
- Analytics and observability vendors
- Cache/coordination store until measured need

Decision standard

Each implementation ADR records context, decision, alternatives, consequences, security/privacy effects, reversibility, and evidence. Vendor selection may not weaken the approved architecture boundaries.

Phase 4 Approval Gate

STATUS: REVIEW REQUIRED

Approval questions

- Does intentional double-booking work as a normal, configurable scheduling mode rather than a workaround?
- Does the segment-and-capacity model match how professionals use processing time, stations, waiting areas, rooms, and equipment?
- Are business policy, scheduling, appointment, booking, payment, theme, audit, integration, and agent responsibilities separated clearly?
- Can web, mobile, voice, AI, and integrations invoke the same actions without direct database access?
- Does theme customization remain expressive without allowing branding to hide or alter essential booking facts?
- Are payment, messaging, events, retries, audit, and failure recovery trustworthy enough to proceed to data design?
- Is the modular-monolith starting posture appropriate for v1?

Approval effect

Approving Phase 4 authorizes Phase 5 Database Architecture: conceptual and physical schema, keys, relationships, constraints, indexes, account isolation, RLS, scheduling conflict protection, policy versioning, audit structures, outbox tables, theme configuration, retention, and migrations.

Not authorized by this approval

- Production implementation
- Final vendor selection
- Phase 6 API contract approval
- Phase 7 screen-design approval
- Launch or production-data processing

Review gate

Can a new interface be added, a provider replaced, or an automation disabled without rewriting or bypassing Aesthenda's booking, policy, payment, or client logic?